

Efficient Log Parsing Method Based on Log Locality Features

Jigang Wen, Chenying Ma, Kun Xie*, Shiming He, Shuyu Pei,
Yuxiang Chen, Wei Liang, and Quan Feng

Abstract: Log parsing is indispensable for system maintenance, converting unstructured log data into structured formats (log templates) for further log compression and anomaly detection. The effectiveness of log parsing relies on the efficiency of two key processes: template extraction and log matching. Traditional methods, however, suffer from slow pairwise comparisons for template extraction and the tedious, non-scalable sequential approach for template matching. Our research has uncovered two opportunities for optimization based on two log locality characteristics: logs from the same template tend to cluster sequentially, and there is a limited variety of templates used within given timeframes. To exploit these opportunities, we propose the Multi-Logs Template Extraction (MLTE)-Cache framework. MLTE-Cache leverages the MLTE algorithm to enhance the efficiency of template extraction by grouping similar logs and processing them in batch mode. Furthermore, the framework utilizes a cache-assisted proximity matching algorithm to accelerate the log matching procedure. Through comprehensive experiments on open-source datasets, the MLTE-Cache framework has proven highly effective, maintaining a high level of accuracy while delivering a 37% improvement in efficiency.

Key words: log parsing; log locality patterns; log template extraction

1 Introduction

1.1 Background and motivation

Effective system operations are crucial for the longevity and reliability of computer systems. Computer logs play a vital role in these processes by providing detailed records of system behavior. For instance, analyzing logs can reveal the reasons behind the abrupt termination of tasks on supercomputing platforms.

Traditionally, system operations rely on domain experts, keyword extraction, and rule-based scripts^[1]. However, the emergence of supercomputers, distributed systems, cloud services, and the Internet of Things (IoT) has exponentially increased the volume and complexity of log data. This increase has overwhelmed traditional manual management methods. Additionally, the diversity in log formats, originating from various systems and users, challenges the effectiveness of rule-based tools for operation and

-
- Jigang Wen, Yuxiang Chen, and Wei Liang are with School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan 411201, China. E-mail: wenjigang@hnust.edu.cn; chen-yuxiang@hnust.edu.cn; wliang@hnust.edu.cn.
 - Chenying Ma, Kun Xie, and Shuyu Pei are with College of Computer Science and Electronics Engineering, Hunan University, Changsha 410082, China, and also with Key Laboratory of Fusion Computing of Supercomputing and Artificial Intelligence of Ministry of Education, Changsha 410082, China. E-mail: machenyang@hnu.edu.cn; xiekun@hnu.edu.cn; peishuyu@hnu.edu.cn.
 - Shiming He is with School of Computer Science and Communication Engineering, Changsha University of Science and Technology, Changsha 410114, China. E-mail: smhe_cs@csust.edu.cn.
 - Quan Feng is with Hunan Vanguard Group Co. Ltd., Changsha 410100, China. E-mail: jgxyfq@126.com.

* To whom correspondence should be addressed.

Manuscript received: 2024-09-02; revised: 2024-09-26; accepted: 2024-10-01

maintenance.

In light of these challenges, AIOps^[2] emerges to combine Information Technology (IT) operations with artificial intelligence. By collecting vast amounts of data from IT infrastructure and using machine learning, AIOps facilitates the automatic detection^[3–6], anomaly analysis^[7, 8], and resolution of system issues in real time. Log parsing stands out as a foundational stage in AIOps, serving to transform unstructured log data into structured formats, suitable for further log analysis^[9, 10], log compression^[11], visualization, log anomaly detection, and more.

Log messages, generated by source code statements such as “logging.info” and “logging.error”, play a critical role in documenting system operations and aiding in error diagnosis^[12, 13], as well as network monitoring^[14]. Log parsing’s main objective is to categorize these messages and distill a generalized template for each category, as illustrated in Fig. 1.

Current log parsing methodologies^[15] are divided into offline and online parsing from the standpoint of real-time analysis. Offline approaches, like SLCT^[16], LFA^[17], and LogCluster^[18], require full dataset for parsing and are not suitable for real-time scenario. In contrast, online algorithms, such as Drain^[19], which uses a fixed-depth tree to measure log similarity, and Spell^[20], which utilizes an expedited Longest Common Subsequence (LCS), can process logs in real time. With advancements in natural language processing^[21], the field has also seen development in text vectorization-based log parsing methods. Despite advances, two main limitations afflict current log parsing methods:

- Template extraction efficiency is hampered by the need for pairwise log message comparison.
- Log matching is inefficient, as it sequentially compares each entry against numerous templates, which is not scalable for large template libraries.

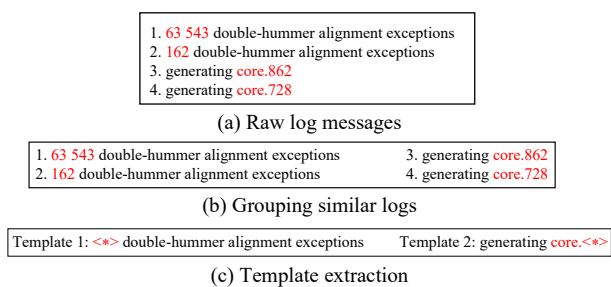


Fig. 1 Example of log parsing.

1.2 Our contribution

From experimental studies, our research uncovers two notable log locality patterns in computer log behaviors:

- Logs sharing the same structural template often appear in adjacent messages.
- Log messages typically use a limited number of templates within a specific time frame.

We introduce the Multi-logs Template Extraction (MLTE)-Cache framework, incorporating the novel MLTE algorithm, designed to streamline template extraction. We also propose a cache-assisted proximity matching algorithm to enhance log message matching efficiency.

The MLTE algorithm exploits the pattern of contiguous log messages sharing identical templates. It accelerates template generation by batching log messages and extracting templates simultaneously for the entire batch. This is complemented by a two-tiered template storage structure, with an extensive library and a supplementary cache, that prioritizes recently used or created template indices.

The cache-assisted proximity matching algorithm expedites matching by first consulting the cache before the larger template library if necessary.

Our paper’s contributions are threefold:

- We introduce the MLTE algorithm, which extracts common phrases from batched logs and structures a hierarchical storage system comprising a template library and cache, enhancing template extraction speed.
- We develop a proximity log matching algorithm that uses a cache system to prioritize recent templates, significantly improving template matching speed.
- We conduct extensive testing on public datasets, validating our algorithm’s effectiveness and efficiency, achieving up to a 37% increase in log parsing speed.

The structure of the paper is as follows: Section 2 introduces related work, Section 3 discusses the experiment on log data locality, Section 4 presents MLTE-Cache log parsing and matching. Section 5 covers experimental setup and analysis, and Section 6 serves as the conclusion.

2 Related Work and Discussion

2.1 Related work

Log parsing has evolved significantly over the years, with numerous algorithms contributing to the field’s advancement. Initially, log parsing relied on frequent itemset mining techniques, exemplified by algorithms

such as SLCT^[16], LFA^[17], and LogCluster^[18]. These methods hinge on the principle that commonly occurring words in logs are typically constants. By traversing the entire log data to identify frequent items, these algorithms group logs and extract templates. However, the necessity for a full data traversal renders these algorithms inherently offline.

Subsequent research has framed log parsing as a clustering challenge, leading to the development of algorithms, like LKE^[10], LogSig^[22], and LogMine^[23], alongside online solutions, such as SHISO^[24] and LenMa^[25]. LKE^[10] clusters log statements by measuring similarity with string edit distance, while LogMine^[23] uses custom distance formulas and a distributed platform to improve efficiency. LogSig^[22] employs a k-means-like approach, categorizing logs into classes based on message signatures. SHISO^[24] dynamically clusters logs using a structured tree built from log messages, and LenMa^[25] categorizes logs based on the length of words, enabling online parsing.

Beyond these methods, recent innovations include algorithms like Spell^[20], Drain^[19], IPLoM^[26], and LPV^[27]. Spell^[20] utilizes the LCS^[28] to gauge log similarity and facilitates online streaming log parsing. Drain^[19] introduces a fixed-depth parse tree for categorizing logs, whereas IPLoM^[26] iteratively classifies logs through a hierarchical strategy. LPV^[27], drawing inspiration from natural language processing, vectorizes log text for classification. Despite their efficacy, text comparison-based methods, such as Drain and Spell face challenges related to efficiency.

2.2 Discussions

Our observations indicate two notable log data characteristics: (1) Log messages of the same template often appear consecutively, and (2) within a brief period, log messages typically involve only a small subset of templates. To improve the efficiency, we leverage these insights to propose the MLTE-Cache framework. Comparing with the above works, MLTE-Cache is the first work of finding the log locality and taking full advantages of it with cache. When log parsing, we extract common phrases from batched logs and store the log template with a hierarchical storage system.

3 Experiment on Log Data Locality

This study investigates the spatial locality of log data through an empirical analysis of consecutive log

messages from various datasets. We randomly sample 100 consecutive log messages from each dataset and visualize their distributions in scatter plots, where the x-axis denotes the log ID and the y-axis corresponds to the template code. For example, E227 represents the 227th log template under the corresponding dataset.

Figure 2 presents the sampling results from open-source BGL, HDFS, and HPC datasets, revealing two prominent features of log data locality:

- Log messages belonging to the same template often appear consecutively. For instance, in Fig. 2a, the BGL dataset exhibits a continuous sequence of 100 log messages (log IDs from 3 297 042 to 3 297 141) corresponding to the template E261 “No power module <*> found on link card”.

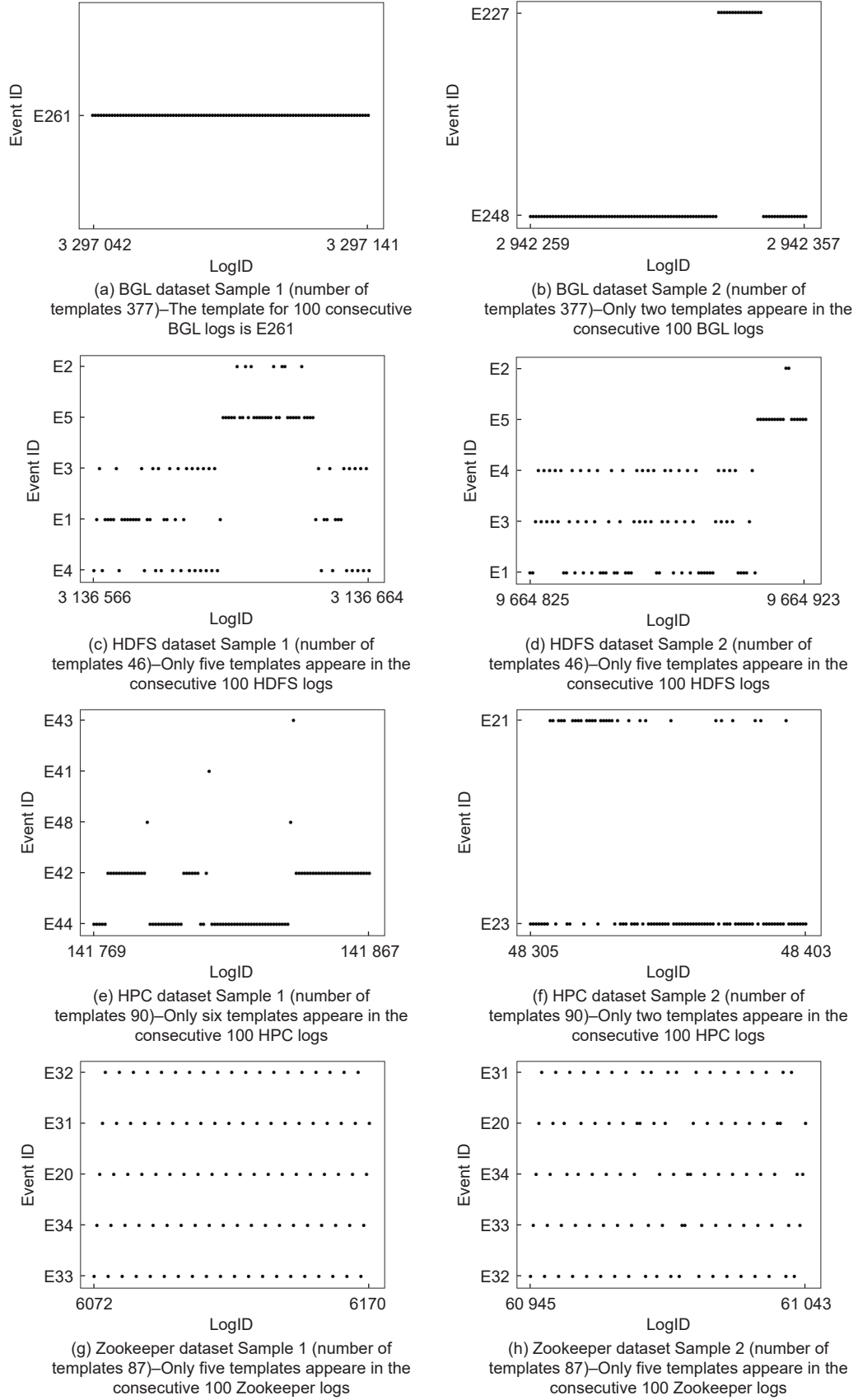
- Within a specific time period, log messages typically involve only a limited subset of templates. In Fig. 2c, a sequence of 100 log messages (log IDs from 3 136 566 to 3 136 664) from the HDFS dataset includes just five out of 46 possible templates. These templates are E1 “Receiving block <*> src: /<*>dest: /<*>”, E2 “BLOCK NameSystem.allocateBlock: <*>”, E3 “PacketResponder <*> for block <*> <*>”, E4 “Received block <*> of size <*> from /<*>”, and E5 “BLOCK NameSystem.addStoredBlock: blockMap updated: <*> is added to <*> size <*>”.

The MLTE-Cache framework is meticulously crafted to harness the observed locality characteristics within log data, thereby improving the efficiency of template extraction and log matching. This framework integrates two core components:

(1) **MLTE algorithm:** Utilizing the tendency of log messages with the same template to occur in sequence, the MLTE algorithm batches these messages to perform concurrent template extraction. It further enhances efficiency by managing a cache that stores indices of templates that appear frequently or have been recently added.

(2) **Cache-assisted matching algorithm:** Addressing the second observed characteristic, where log messages within a short time frame are dominated by a few templates, this algorithm optimizes the log matching process. It achieves this by giving precedence to templates in the cache, thus expediting the search for the correct template match among a limited set of frequently used templates.

By leveraging these innovative strategies, the MLTE-Cache framework significantly accelerates the log

**Fig. 2** Log message template distribution across datasets.

parsing process.

4 MLTE-Cache Log Parsing and Matching

This section introduces the MLTE-Cache, a method streamlined for enhanced log template extraction and matching efficiency. Figure 3 presents the comprehensive model of MLTE-Cache, which unfolds in two principal phases: template extraction and log matching.

- **Template extraction phase:** The MLTE-Cache begins with the template extraction phase. A subset of log messages is extracted from the dataset and partitioned into batches according to log length, ensuring uniformity within each batch. The bespoke MLTE algorithm is subsequently applied to each batch to identify and extract common word sequences. Upon reaching the predefined commonality threshold, templates are generated, and the template library along with the cache is updated. In cases where the threshold is not met, the batch is subjected to a log splitting algorithm, which operates on the least frequent word types at random positions, prompting a re-extraction of common words. This phase culminates with the establishment of the initial template library and Cache, which retains indices of the most recent and newly created templates for subsequent log matching.

- **Log matching phase:** During the log matching phase, MLTE-Cache prioritizes the cache, attempting to align each log message with templates from the recent cache. A failure to find a match in the cache leads to a search within the broader template library.

Unmatched log messages are then designated as unique templates and incorporated into the template library. The process concludes with an update to both the template library and the cache, reflecting the latest matching outcomes.

4.1 Basic concepts

The methodology employed in this study is underpinned by several key concepts:

- (1) **Unique words:** These are the distinct words obtained from a collection by eliminating duplicates. For instance, the collection {iar, double, 0xffffee01, iar} yields a unique word set of {iar, double, 0xffffee01}, with the count being three.

- (2) **Wildcard:** Represented by the symbol $<*>$, a wildcard acts as a placeholder for variable elements within log messages, enabling the abstraction of log templates.

- (3) **Log length:** This is the total count of words present in a log message.

- (4) **Template definition:** A log template is defined by several attributes:

- **Template length:** The combined count of constants and wildcards in the template.

- **Template content:** The actual words and wildcards comprising the template.

- **Template constant positions:** The indices within the template content array that correspond to constant words, excluding the wildcard placeholders.

- **Log IDs:** A collection of log message IDs that are categorized under the template.

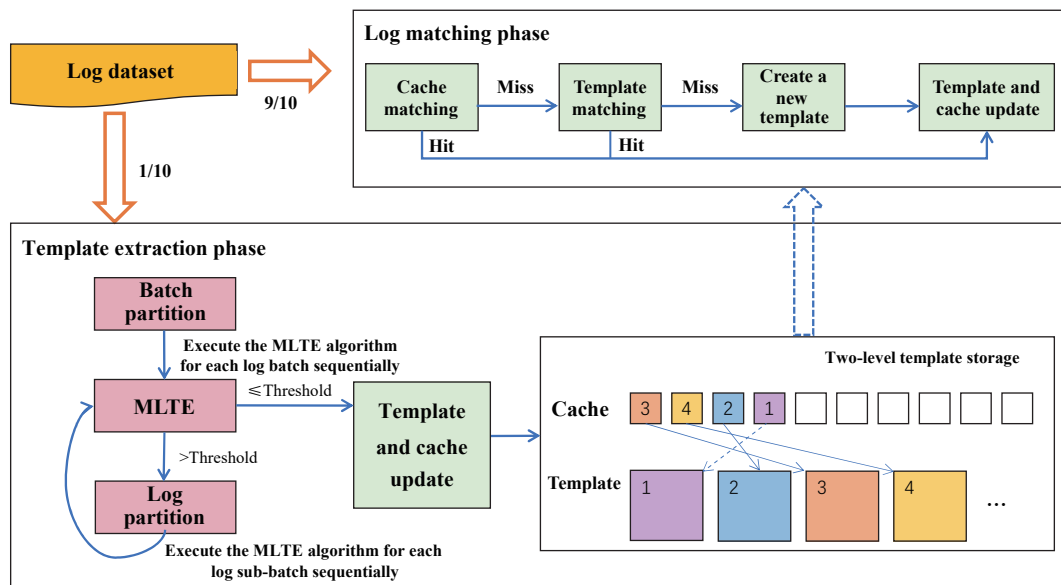


Fig. 3 Overview of MLTE-Cache log parsing method.

Figure 4 demonstrates these components using a sample template structure. For the template “node <*> detected a failed network <*> via interface alt0”, the structure encompasses the template length (10 words), the content (words array derived by splitting the template string at spaces), the constant positions (index array), and the associated log IDs.

4.2 MLTE template extraction algorithm for log datasets

The MLTE algorithm aims to distill one or more templates from a collection of log messages within a batch. This algorithm follows a structured approach, delineated into three key steps:

- (1) Extraction of common words from the set of log messages in a batch.
- (2) Assessment against a predetermined threshold to determine if sufficient common words exist to form a template. If the threshold is met, a template is extracted for the batch.
- (3) Division of the batch into sub-batches for further extraction in cases where the threshold is not met.

4.2.1 Multi-logs common words extraction algorithm

In computer log analysis, it is common for log messages that share a template to also share the same length. Capitalizing on this observation, the MLTE-Cache first categorizes log messages into separate batches based on log length. The MLTE algorithm, which is specifically tailored for this study, is then applied to extract common words within each batch.

In the structure of log templates, certain word positions are designated as constant, with their respective words remaining unchanged across all messages of the template. In contrast, the words at variable positions are subject to change. For instance, in the template “Receiving block <*> src: <*> dest: <*>”, while the first two words, “Receiving” and “block”, remain consistent, variability is introduced at the third position, with different messages having a

different word in this position.

When processing multiple log messages, the MLTE algorithm examines each word position, identified by index i , to verify the consistency of words across messages. If consistency is found, a common word at that position is confirmed. However, if a word discrepancy is detected at any position i , the algorithm marks it as variable and indicates a failed extraction of a common word.

Figure 5 demonstrates the MLTE algorithm processing various log sequences. In Fig. 5a, the algorithm identifies a lack of commonality at the first word position, as the word from log message 2, “63 561”, differs from that of log message 1, “63 501”. Consequently, the search for a common word at this position is halted. The algorithm then examines the second word across the messages and finds uniformity with the term “double-hummer”, which is then recognized as a common word at position two. It continues to identify “alignment” and “exceptions” as common words at the third and fourth positions, respectively, resulting in successful extraction for the message batch. In stark contrast, Fig. 5b shows a scenario where different words at each position prevent the extraction of any common words for the batch.

In the MLTE algorithm, constant positions require traversing all log messages in the batch. However, for variable positions, in the best-case scenario, the algorithm only needs to compare two log messages to determine the differences; in the worst-case scenario, for N messages, it requires $N - 1$ comparisons.

Upon extracting common words from the batch, the MLTE algorithm evaluates whether the count meets a pre-set threshold. Meeting this threshold indicates successful common word extraction, leading to the generation of log templates for the batch and updates to the template library and cache. Failure to meet the threshold prompts the algorithm to employ a log splitting strategy, dividing the batch into smaller sub-batches using the minimum number of word types at

Template length	10
Template content	[node, <*>, detected, a, failed, network, <*>, via, interface, alt0]
Template constant positions	[0, 2, 3, 4, 5, 7, 8, 9]
Log IDs	100, 101, 102, 103, ...

Fig. 4 Example of a log template data structure.

Word position:	0	1	2	3
Log message:	1. 63 501	double-hummer	alignment	exceptions
	2. 63 561	double-hummer	alignment	exceptions
	3. 63 471	double-hummer	alignment	exceptions
	4. 63 626	double-hummer	alignment	exceptions
Common word:	-	double-hummer	alignment	exceptions

(a) Example of successful common word extraction by the MLTE algorithm

Word position:	0	1	2	3
Log message:	1. 63 501	double-hummer	alignment	exceptions
	2. iar	003a9260	dear	00efde18
	3. 63 471	double-hummer	alignment	exceptions
	4. 63 626	double-hummer	alignment	exceptions
Common word:	-	-	-	-

(b) Example that the MLTE algorithm fails to extract common words

Fig. 5 Operational illustrations of the MLTE algorithm.

random positions in Section 4.2.2. The threshold is defined as

$$\text{threshold} = \begin{cases} \frac{L}{2}, & \text{if } L \leq 20 \text{ and } L\%2 = 0; \\ \frac{L+1}{2}, & \text{if } L \leq 20 \text{ and } L\%2 \neq 0; \\ \frac{L}{4}, & \text{if } L > 20 \end{cases} \quad (1)$$

where L is the number of words.

In Fig. 5a, a batch of four log messages contains common words, such as “double-hummer”, “alignment”, and “exceptions”, totaling three common words, which surpasses the threshold of 2 based on the log length. Consequently, a template “/<*>double-hummer alignment exceptions” is constructed. In contrast, Fig. 5b shows four log messages without any common words, failing to meet the threshold of 2 and resulting in unsuccessful template extraction.

When the extraction of common words is successful, the MLTE algorithm generates log templates for the batch as follows. Let “log1” represent the first log message in the batch and “Ctokens” represent the set of extracted common words:

- Initialize an empty log template data structure, denoted as Tpl. Set the length of Tpl to match that of log1. Initialize the template content and the array of constant positions within Tpl as empty. Include all log message IDs from the batch in Tpl’s metadata.
- Iterate through each word in log1, denoted by log1[i]. Check for the existence of log1[i] in Ctokens.
- If log1[i] is found in Ctokens, append log1[i] to the template content of Tpl and add the index i to the array of constant positions within Tpl.
- If log1[i] is not present in Ctokens, insert a wildcard symbol, denoted by “<*>”, into the template

content of Tpl, indicating a variable position.

4.2.2 Log splitting algorithm based on random positions and the minimum number of word types

The primary issue in template extraction using the MLTE algorithm occurs when a batch contains log messages that do not conform to a single template, thwarting the extraction of a unified set of common words that meet pre-established threshold criteria. The diversity of the messages, potentially corresponding to multiple templates, complicates the identification and extraction of the correct templates.

This challenge is exemplified in Fig. 5b, where a batch of four log messages yields no common words because they align with different templates. Specifically, log messages 1, 3, and 4 are associated with the template “/<*> double-hummer alignment exceptions”, while log message 2 follows a distinct template, “iar <*> dear <*>”. Circumstances like these necessitate the segmentation of the batch into smaller sub-batches to facilitate the extraction of the respective templates.

To remediate this issue, we suggest dividing the batch into smaller sub-batches for independent processing and extraction of appropriate templates corresponding to each group of log messages. This strategic division is imperative for managing logs that do not consistently align with a single template, ensuring the successful completion of the template extraction process.

Algorithm 1 outlines the log splitting process, which relies on random position selection and the minimum number of word types present at each position. A position with only one word type denotes a constant across all logs, while a position with a word type count equal to N indicates that each log contains a unique word. This dichotomy results in two distinct scenarios: (1) each log corresponds to a unique template, necessitating individual sub-batch treatment; (2) logs are associated with fewer than N templates, exhibiting variability at those positions. Consequently, the algorithm selects three random positions and identifies the one with the smallest yet non-unique word type count as the splitting criterion for dividing into smaller sub-batches.

Figure 6 demonstrates the log splitting algorithm operationalized through an example. When positions 0, 1, and 2 are randomly selected, as depicted in Fig. 6b, the algorithm identifies position 2 as the splitting point

Algorithm 1 Log segmentation algorithm based on random position and minimum word type diversity**Require:** Logs (a batch consisting of LogNum logs, each with a length of LogLen)**Ensure:** subBatches (multiple sub-batches generated from logs following partitioning, initially empty)

```

1: Shuffle RArray  $\leftarrow$  np.arange(LogLen); # Generate and
   randomize an array with values ranging from 1 to LogLen
2: Tss  $\leftarrow$  [ ]; # Initialize an array to track positions with
   minimal word type diversity
3: for pos in RArray do
4:   tokens  $\leftarrow$  { }; # Create a dictionary to record word types
   at position pos, with words as keys and log ID collections
   as values
5:   for log in Logs do
6:     Assess if the word at position pos in log, denoted
       tokenp, is a key in tokens;
7:     If “yes”, append the ID of log to the value array of
       tokens [tokenp];
8:     If “no”, insert a new key tokenp into tokens and link it to
       the ID of log;
9:   end for
10:  If the size of tokens falls between 1 and LogNum, add
    tokens to Tss;
11:  Terminate the loop if Tss contains the pre-set number of 3
    elements;
12: end for
13: Select splitBy from Tss, which is the element with the least
    word type diversity, and denote the number of word types as
    mintokens;
14: if mintokens < LogNum then
15:   Bundle logs associated with each key in splitBy into a
    sub-batch;
16: else
17:   Assign each log in Logs to a separate sub-batch;
18: end if
19: return subBatches

```

due to the occurrence of two distinct word types: “double-hummer” and “003a9260”. This distinction leads to the formation of two sub-batches: (1, 2, 3, 4) and (5, 6, 7, 8). Conversely, if positions 0, 2, and 3 are chosen, as shown in Fig. 6c, it is position 3 that is selected as the splitting point, similarly resulting in a bifurcation into two sub-batches: (1, 2, 3, 4) and (5, 6, 7, 8).

Post-segmentation, the MLTE algorithm proceeds to extract common words from each sub-batch, thereby advancing the template extraction process. It is noteworthy that the log splitting algorithm is not a one-off application; it may be iteratively applied to the same batch. If the initial division fails to elicit common

1. 63501 double-hummer alignment exceptions
2. 63561 double-hummer alignment exceptions
3. 63471 double-hummer alignment exceptions
4. 63626 double-hummer alignment exceptions
5. iar 003a9260 dear 00efde18
6. iar 003a9260 dear 00fcdd58
7. iar 003a9260 dear 00fccbf8
8. iar 003a9260 dear 00efc6c8

(a) Sequence of logs within the same batch

Position	Type of words	Unique word
0	5	63501,63561,63471,63626,iar
1	2	double-hummer,003a9260
2	2	alignment,dear

(b) Segmentation with randomly selected positions 0, 1, and 2

Position	Type of words	Unique word
0	5	63501,63561,63471,63626,iar
2	2	alignment,dear
3	5	exceptions,00efde18,00fcdd58,00fccbf8,00efc6c8

(c) Segmentation with randomly selected positions 0, 2, 3

Fig. 6 Illustration of the log segmentation algorithm.

words within a subbatch, the algorithm is re-engaged with those specific sub-batches lacking common word extraction. This iterative cycle is repeated until it culminates in the successful extraction of common words, or when each sub-batch is distilled down to a singular log message, which then inherently serves as its own template.

In summary, the log splitting algorithm depicted in Fig. 6 and described in Algorithm 1 showcases a dynamic and adaptable approach to segregating log messages into sub-batches for more effective template extraction. This methodical division is predicated on the identification of word type diversity across randomly selected positions, ensuring that logs are grouped in a manner conducive to distinguishing and extracting the underlying templates.

4.2.3 Template library and cache update

After the successful extraction of a template, denoted by T_i , for a batch or subbatch, MLTE-Cache proceeds to update the template library and record the new template or the index of the matched template in the cache, creating a two-tiered storage structure. The update procedure is as follows:

- Access the template library using the index retrieved from the cache. Check for an existing template, T_{old} , that fully matches T_i . If found, update

both the template library and cache accordingly.

- If no match is found in the cache, through the template library to find a template, T_{old} , that matches T_i exactly. Update both the template library and cache upon finding a match.

- Give priority to exact matches when comparing templates. In the absence of an exact match, perform common word extraction between T_i and each potentially matching template, T_{old} . If T_i and T_{old} share a sufficient number of common words above a certain threshold, merge them into a new template, T_{new} . This accounts for instances where variable positions have coincidentally become constants in T_i during previous template extractions.

- If T_i does not match any template in the library and does not satisfy the merging conditions from the previous step, add it as a new template to the library. Also, update the cache with this new template. When T_i has an exact match, T_x , in the library, append the log ID sequence of T_i to that of T_x . Update the cache by placing the index of T_x at the top. If the cache reaches maximum capacity, the least recently used index is removed to make space for the new entry.

During template merging, the update process involves converting non-matching constants between T_i and T_{old} into variables and removing their positions from the sequence of constant positions. Then, the log

ID sequence of T_i is integrated into that of T_{old} . Figure 7 exemplifies a template merging case. Although T_i and T_{old} do not have identical contents, they share a significant number of common words—[node, detected, a, failed, network, via, interface]—which surpasses the threshold of 5, thus allowing their merger into a unified template.

4.3 Cache-accelerated log matching algorithm

In the template extraction phase, a two-tiered storage structure is established: the cache, which holds indices of recently accessed templates up to a capacity of CacheSize, and the template library, which retains all the templates. This structure is leveraged in the MLTE-Cache to expedite the log matching process. For each incoming log message, the algorithm first checks the cache for a match. If found, the process concludes; if not, the algorithm moves on to search the template library. A match in the library also concludes the process. If no match is found, the log message is added as a new template to the library. Template matching involves the following steps:

- (1) Check if the log message and the template have equal lengths. If they do not match, proceed to the next template. If they do, move to the next comparison step.

- (2) Verify that each word at the constant positions of the template matches its counterpart in the log

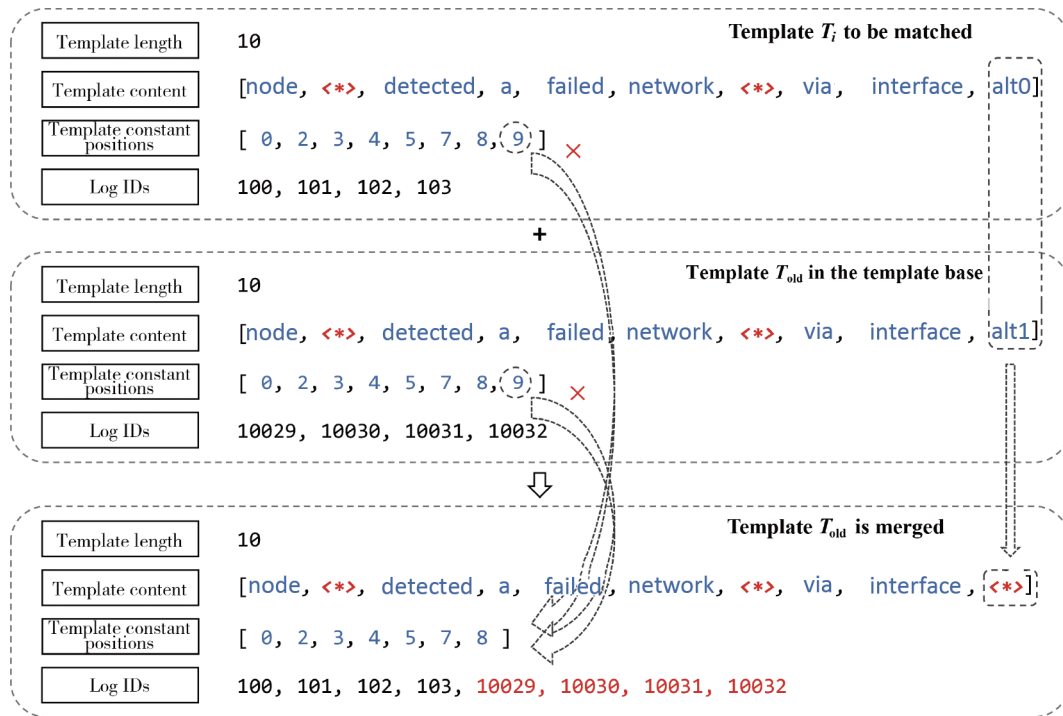


Fig. 7 Example of template mergence.

message. If all words match, the process concludes successfully. In case of any mismatches, count the number of common words. If this number meets a predefined threshold, note the index of the template with the highest common word count for later use.

(3) Should the log message fail to match any cached or library template, refer to the noted index from Step 2. If it exists, conclude the process. If not, add the log message as a new template and update both the library and cache.

(4) Update the template library and cache post-matching. Cache updates involve promoting the indices of matched or new templates to the front. The template library can be updated in one of three ways:

(a) For a log message that fully matches an existing template, append its ID to the log IDs sequence of the matched template.

(b) For a log message that does not fully match but shares a maximum number of common words above the threshold with an existing template, merge it with the existing template T_{old} and add its ID to T_{old} 's log IDs sequence.

(c) For a log message that does not match any template nor meets the threshold for common words, add it as a new template to the library.

Figure 8 depicts the template library and cache updates following various log matching outcomes. Figure 8a shows the initial state, while Figs. 8b–8d illustrate updates of Figs. 8a–8c according to the fourth step mentioned above, respectively.

The cache serves to decrease the number of matching operations required, notably reducing matching time, especially in worst-case scenarios where iterating through the entire template library might otherwise be necessary.

4.4 Algorithm analysis

MLTE-Cache comprises two primary steps: template extraction and log matching. The MLTE algorithm is designed to optimize the template extraction process, while a cache-based nearest matching algorithm is used to expedite the log matching step. This section analyzes the algorithmic complexity of both the template extraction and log matching steps, as well as offering an overall assessment of the model.

4.4.1 Template extraction algorithm analysis

Let l represent the length of a log message, with c and v denoting the counts of constants and variables, respectively, and N signifying the number of log

entries in a batch, satisfying $c + v = l$. When applying the MLTE algorithm to N log messages in a batch, it conducts $c \times (N - 1) + v$ word comparisons. Specifically, the term $c \times (N - 1)$ accounts for the comparisons at all constant positions, where each constant position i requires $N - 1$ comparisons to confirm its invariability across messages. Conversely, the term v refers to the comparisons at variable positions, where the content of the messages may differ. In most cases, a single comparison is sufficient to establish whether there is variability at each variable position.

In contrast, traditional methods would necessitate $l \times (N - 1)$ word comparisons, which simplifies to $c \times (N - 1) + v \times (N - 1)$, since each log message in the batch must be compared with all others. Initially, to extract template T , the first two log messages are compared, which requires l comparisons. Subsequently, to confirm that the remaining log messages adhere to template T , each message is compared with the template, resulting in an additional $l \times (N - 2)$ comparisons. Thus, traditional methods involve a more exhaustive comparison process.

Therefore, for a batch with N log messages, MLTE reduces the word comparison count during template extraction by $v \times (N - 2)$ times.

4.4.2 Log matching algorithm analysis

Table 1 displays the cache hit ratios achieved by MLTE-Cache for various datasets. The cache hit ratios are nearly 100% for the BGL and HDFS datasets, and around 97% for the HPC and Zookeeper datasets. This indicates that the cache-based nearest matching algorithm in MLTE-Cache generally only needs to match against templates in the cache when matching log messages, leading to a time complexity of $O(c \times m \times n)$. Due to the introduction of cache, the additional memory space of our solution is only related to the cache space $O(c)$. Here, c denotes the cache size, m the log length, and n the number of log messages. Since c is a constant value of 10 and m is significantly smaller than n , the time complexity of log matching is effectively $O(n)$.

5 Experimental Setup and Analysis

5.1 Experimental configuration

The MLTE-Cache model is implemented and tested on an Honor laptop with the specifications detailed below:

- Operating system: Windows 10 Professional (64-bit)

Cache 3 2 1

Template base

Index	Template length	FileBody	Constant position	Log IDs
1	4	<*> TLB error interrupt	1,2,3	1,2,3,...
2	2	generating <*>	0	17,18,23,...
3	4	L3 major internal error	0,1,2,3	11,22,...

(a) Initial template library and cache state

Cache 1 3 2

Template base

Index	Template length	FileBody	Constant position	Log IDs
1	4	<*> TLB error interrupt	1,2,3	1,2,3,54,...
2	2	generating <*>	0	17,18,23,...
3	4	L3 major internal error	0,1,2,3	11,22,...

(b) Log “54: data2 TLB error interrupt” is matched

Cache 3 1 2

Template base

Index	Template length	FileBody	Constant position	Log IDs
1	4	<*> TLB error interrupt	1,2,3	1,2,3,54,...
2	2	generating <*>	0	17,18,23,...
3	4	<*> major internal error	1,2,3	11,22,67,...

(c) Log “67: L1 major internal error” is matched

Cache 4 3 1

Template base

Index	Template length	FileBody	Constant position	Log IDs
1	4	<*> TLB error interrupt	1,2,3	1,2,3,54,...
2	2	generating <*>	0	17,18,23,...
3	4	<*> major internal error	1,2,3	11,22,67,...
4	3	chip select 0	0,1,2	72

(d) Log “72: chip select 0” is matched

Fig. 8 Template library and cache update scenarios.**Table 1** Cache hit ratio under various datasets.

Dataset	Value
BGL	99.5
HDFS	99.9
HPC	97.5
Zookeeper	97.6

- Processor: AMD Ryzen 53550H
- Memory: 8 GB RAM
- Programming language: Python 3.5
- Development environment: JetBrains PyCharm

2019.3 (64-bit)

5.2 Datasets, comparative algorithms, and evaluation metrics

(1) Datasets: We utilized four open-source datasets for our experiments: BGL^[29], HPC, HDFS^[30], and Zookeeper. The first two datasets are from supercomputing environments, while the latter two originate from distributed systems. Table 2 presents a summary of these datasets. All datasets are available for download from Loghub^[31]. Further, log datasets with 2000-sized samples and pre-labeled parsing

results are accessible at logpai/logparser^[32]. Given the limited scale of the 2000-sized sample datasets, which may introduce some degree of randomness to the results, our experiments focus solely on the four comprehensive datasets with complete parsing labels.

(2) Comparative algorithms: We benchmark our model against three well-established log parsing algorithms: Drain, ILoM, and Spell.

(a) Drain^[19]: Drain employs a fixed-depth parsing tree to categorize logs and extract templates. The first layer of the tree captures the length of log messages, while the leaf nodes hold the log templates and the set of log message IDs corresponding to each template. Drain is suitable for online execution.

(b) ILoM^[26]: ILoM uses a three-step hierarchical and iterative method for log classification and template generation. The divisions are based on token count, token position, and word pair relationships, leading to template extraction for each partition. ILoM operates as an offline algorithm.

(c) Spell^[20]: Spell leverages an accelerated Longest Common Subsequence method for log classification and template extraction, incorporating inverted index technology to enhance the matching speed. Spell functions as an online parser.

(3) Evaluation metrics: To assess the efficacy of the models, we utilized the following metrics: Accuracy^[33] and the F1 score^[34, 35].

(a) Accuracy: This metric is defined based on the number of templates, N , with log messages to be accordingly partitioned into N groups. A log message's extraction is deemed successful if it is correctly allocated to the group corresponding to its template. The accuracy metric specifically evaluates the consistency of grouping for logs associated with the same template, rather than the exact match between the extracted and the actual template.

(b) F1-score: The F1-score, representing the harmonic mean of Precision and Recall, is given by

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2)$$

where Precision and Recall are defined as follows:

Table 2 Log data set.

Dataset	Size (bit)	Number of templates
BGL	5.95×10^9	377
HPC Cluster	2.68×10^8	90
HDFS	1.25×10^{10}	46
Zookeeper	8.34×10^7	87

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (3)$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (4)$$

where True Positive (TP) is the count of log messages correctly grouped under the same template, False Positive (FP) is the count of log messages from disparate templates mistakenly grouped together, and False Negative (FN) is the count of log messages from the same template that are wrongly classified into different groups.

5.3 Experimental results analysis

5.3.1 Parsing accuracy comparison

Table 3 presents the comparison of parsing accuracy across different datasets, with the highest accuracy for each dataset highlighted in bold. MLTE-Cache exhibits the highest accuracy on the BGL, HDFS, and HPC datasets. However, for the Zookeeper dataset, the Drain algorithm outperforms the others. The decreased accuracy of MLTE-Cache in comparison to Drain can be attributed to MLTE-Cache's merging of similar templates, such as conflating "initLimit set to <*>" with "maxSessionTimeout set to <*>" and "tickTime set to <*>", into a single template "<*> set to <*>".

Table 4 shows the F1-scores, where MLTE-Cache achieves the best results on each dataset. The F1 scores for all algorithms are nearly identical, especially on HDFS and Zookeeper, where the F1-score for each algorithm is 1.

5.3.2 Parsing efficiency comparison

The efficiency comparison of log parsing is shown in Table 5. Taking the BGL dataset as an example, the time required to run the MLTE cache log parsing algorithm is 771 s, which is much less than other methods. MLTE-Cache demonstrates superior performance on large datasets due to its cache-based matching algorithm, which reduces the need to traverse the entire template library by leveraging a cache for frequently accessed templates, thereby enhancing

Table 3 Log parsing accuracy.

Dataset	Method			
	Drain	ILoM	Spell	MLTE-Cache
BGL	0.88	0.89	0.68	0.92
HDFS	1.00	0.96	0.81	1.00
HPC	0.73	0.68	0.53	0.73
Zookeeper	0.97	0.96	0.96	0.96

Table 4 F1-score.

Dataset	Method			
	Drain	IPLoM	Spell	MLTE-Cache
BGL	1.00	1.00	0.96	1.00
HDFS	1.00	1.00	1.00	1.00
HPC	0.90	0.89	0.90	0.90
Zookeeper	1.00	1.00	1.00	1.00

efficiency, especially when processing a large number of log messages or templates.

5.3.3 Ablation experiment

The ablation experiment, detailed in Table 6, compares the parsing efficiency of the proposed method with and without the use of a cache. The “MLTE” row represents the method’s performance without cache utilization, while “MLTE-Cache” indicates the method’s performance with the cache. The results highlight significant efficiency gains from cache usage, particularly in processing larger datasets, such as BGL, HDFS, and HPC. For the BGL dataset, which contains a larger number of templates, the cache’s role in storing indices of recently accessed templates and prioritizing cache-based template matching results in an efficiency improvement of up to 37%.

6 Conclusion

In system maintenance, the ability to parse logs efficiently is paramount for the rapid detection of anomalies and ensuring system stability. Traditional log parsing methods often fall short due to their reliance on slow, pairwise comparisons and linear, non-scalable processing for template extraction and log

matching. Building upon insights from empirical research on log characteristics, this paper introduces MLTE-Cache, a cutting-edge algorithmic framework poised to significantly enhance log parsing efficiency. MLTE-Cache unfolds in two primary phases. Initially, the MLTE algorithm innovatively performs template extraction through batch processing, a departure from the conventional one-at-a-time approach. This strategy not only accelerates the extraction process, but also lays the foundation for a robust template library and an associated cache. The cache is strategically engineered to index templates that are freshly created or have been used recently, thus ensuring rapid retrieval of the most pertinent templates. Subsequently, in the log matching phase, MLTE-Cache incorporates a cache-oriented matching algorithm. This component is designed to first consult the cache for potential matches before scanning the extensive template library, capitalizing on the cache’s expedited access feature. Through rigorous experimental evaluation involving four open-source datasets, MLTE-Cache demonstrated remarkable performance improvements over existing log parsing algorithms. It achieved a substantial 37% boost in efficiency while consistently maintaining high accuracy levels.

Acknowledgment

The work was supported by the National Natural Science Foundation of China (Nos. 62025201, 62472167, 62272062, and 62202156), the Hunan Provincial Natural Science Foundation of China (Nos. 2024JJ3014, 2024JJ5165, and 2024JJ6220), the Key Research and Development Program of Hunan Province (No. 2023GK2001), the Joint Key Project of National Natural Science Foundation of China (No. U2468205), and the Science and Technology Innovation Program of Hunan Province (No. 2023RC3139).

References

- [1] D. Lang, Using sec, *Login*, vol. 38, no. 6, pp. 38–43, 2013.
- [2] Y. Dang, Q. Lin, and P. Huang, AIOps: Real-world challenges and research innovations, in *Proc. 2019 IEEE/ACM 41st Int. Conf. Software Engineering: Companion Proc. (ICSE-Companion)*, Montreal, Canada, 2019, pp. 4–5.
- [3] S. He, G. Li, T. Yi, O. Alfarrarj, A. Tolba, A. K. Sangaiah, and R. S. Sherratt, Graph structure learning-based multivariate time series anomaly detection in internet of things for human-centric consumer applications, *IEEE Trans. Consum. Electron.*, doi: 10.1109/TCE.2024.

Table 5 Analytical efficiency (time).

Dataset	Method			
	Drain	IPLoM	Spell	MLTE-Cache
BGL	1200	824	3708	771
HDFS	3300	3460	3930	2152
HPC	85	66	74	69
Zookeeper	16	12	14	12

Table 6 Efficiency (time) of using cache vs. not using cache.

Dataset	Method	
	MLTE	MLTE-Cache
BGL	1270	797
HDFS	2518	2147
HPC	78	68
Zookeeper	12	12

- 3409391.
- [4] S. He, G. Li, K. Xie, and P. K. Sharma, Fusion graph structure learning-based multivariate time series anomaly detection with structured prior knowledge, *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 8760–8772, 2024.
 - [5] Y. Yin, K. Xie, S. He, Y. Li, J. Wen, Z. Diao, D. Zhang, and G. Xie, GraphIoT: Lightweight IoT device detection based on graph classifiers and incremental learning, *IEEE Trans. Serv. Comput.*, doi: 10.1109/TSC.2024.3466854.
 - [6] S. He, Q. Guo, G. Li, K. Xie, and P. K. Sharma, Multivariate time series anomaly detection based on multiple spatio-temporal graph convolution, *IEEE Trans. Instrum. Meas.*, doi: 10.1109/TIM.2024.3493890.
 - [7] K. Xie, X. Li, X. Wang, G. Xie, J. Wen, J. Cao, and D. Zhang, Fast tensor factorization for accurate internet anomaly detection, *IEEE/ACM Trans. Netw.*, vol. 25, no. 6, pp. 3794–3807, 2017.
 - [8] S. Pei, J. Wen, K. Xie, G. Xie, and K. Li, On-line network traffic anomaly detection based on tensor sketch, *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 12, pp. 3028–3045, 2023.
 - [9] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, Detecting large-scale system problems by mining console logs, in *Proc. ACM SIGOPS 22nd Symp. Operating Systems Principles*, Big Sky, MT, USA, 2009, pp. 117–132.
 - [10] Q. Fu, J. G. Lou, Y. Wang, and J. Li, Execution anomaly detection in distributed systems through unstructured log analysis, in *Proc. 2009 9th IEEE Int. Conf. Data Mining*, Miami Beach, FL, USA, 2009, pp. 149–158.
 - [11] K. Rodrigues, Y. Luo, and D. Yuan, CLP: Efficient and scalable search on compressed text logs, in *Proc. 15th USENIX Symp. Operating Systems Design and Implementation*, <https://www.usenix.org/conference/osdi21/presentation/rodrigues>, 2021.
 - [12] W. E. Wong, V. Debroy, R. Golden, X. Xu, and B. Thuraisingham, Effective software fault localization using an RBF neural network, *IEEE Trans. Reliab.*, vol. 61, no. 1, pp. 149–169, 2012.
 - [13] D. Q. Zou, H. Qin, and H. Jin, UiLog: Improving log-based fault diagnosis by log analysis, *J. Comput. Sci. Technol.*, vol. 31, no. 5, pp. 1038–1052, 2016.
 - [14] K. Yamanishi and Y. Maruyama, Dynamic syslog mining for network failure monitoring, in *Proc. 11th ACM SIGKDD Int. Conf. Knowledge Discovery in Data Mining*, Chicago, IL, USA, 2005, pp. 499–508.
 - [15] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, and M. R. Lyu, Tools and benchmarks for automated log parsing, in *Proc. 41st Int. Conf. Software Engineering: Software Engineering in Practice*, Montreal, Canada, 2019, pp. 121–130.
 - [16] R. Vaarandi, A data clustering algorithm for mining patterns from event logs, in *Proc. 3rd IEEE Workshop on IP Operations & Management*, Kansas City, MO, USA, 2003, pp. 119–126.
 - [17] M. Nagappan and M. A. Vouk, Abstracting log lines to log event types for mining software system logs, in *Proc. 2010 7th IEEE Working Conf. Mining Software Repositories*, Cape Town, South Africa, 2010, pp. 114–117.
 - [18] R. Vaarandi and M. Pihelgas, LogCluster—a data clustering and pattern mining algorithm for event logs, in *Proc. 2015 11th Int. Conf. Network and Service Management*, Barcelona, Spain, 2015, pp. 1–7.
 - [19] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, Drain: An online log parsing approach with fixed depth tree, in *Proc. 2017 IEEE Int. Conf. Web Services*, Honolulu, HI, USA, 2017, pp. 33–40.
 - [20] M. Du and F. Li, Spell: Online streaming parsing of large unstructured system logs, *IEEE Trans. Knowledge Data Eng.*, vol. 31, no. 11, pp. 2213–2227, 2019.
 - [21] T. Mikolov, K. Chen, G. Corrado, and J. Dean, Efficient estimation of word representations in vector space, in *Proc. 1st Int. Conf. Learning Representations*, Scottsdale, AZ, USA, 2013, doi: 10.48550/arXiv.1301.3781.
 - [22] L. Tang, T. Li, and C. S. Perng, LogSig: Generating system events from raw textual logs, in *Proc. 20th ACM Int. Conf. Information and Knowledge Management*, Glasgow, UK, 2011, pp. 785–794.
 - [23] H. Hamooni, B. Debnath, J. Xu, H. Zhang, G. Jiang, and A. Mueen, LogMine: Fast pattern recognition for log analytics, in *Proc. 25th ACM Int. Conf. Information and Knowledge Management*, Indianapolis, IN, USA, 2016, pp. 1573–1582.
 - [24] M. Mizutani, Incremental mining of system log format, in *Proc. 2013 IEEE Int. Conf. Services Computing*, Santa Clara, CA, USA, 2013, pp. 595–602.
 - [25] K. Shima, Length matters: Clustering system log messages using length of words, arXiv preprint arXiv: 1611.03213, 2016.
 - [26] A. A. O. Makanju, A. N. Zincir-Heywood, and E. E. Milios, Clustering event logs using iterative partitioning, in *Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, Paris, France, 2009, pp. 1255–1264.
 - [27] T. Xiao, Z. Quan, Z. J. Wang, K. Zhao, and X. Liao, LPV: A log parser based on vectorization for offline and online log parsing, in *Proc. 2020 IEEE Int. Conf. Data Mining*, Sorrento, Italy, 2020, pp. 1346–1351.
 - [28] D. S. Hirschberg, A linear space algorithm for computing maximal common subsequences, *Commun. ACM*, vol. 18, no. 6, pp. 341–343, 1975.
 - [29] P. He, J. Zhu, S. He, J. Li, and M. R. Lyu, An evaluation study on log parsing and its use in log mining, in *Proc. 2016 46th Annu. IEEE/IFIP Int. Conf. Dependable Systems and Networks*, Toulouse, France, 2016, pp. 654–661.
 - [30] S. Messaoudi, A. Panichella, D. Bianculli, L. Briand, and R. Sasnauskas, A search-based approach for accurate identification of log message formats, in *Proc. IEEE/ACM 26th Int. Conf. Program Comprehension*, Gothenburg, Sweden, 2018, pp. 167–177.
 - [31] R. Vaarandi, A data clustering algorithm for mining patterns from event logs, in *Proc. 3rd IEEE Workshop on IP Operations & Management*, Kansas City, MO, USA, 2003, pp. 119–126.
 - [32] M. Nagappan and M. A. Vouk, Abstracting log lines to log event types for mining software system logs, in *Proc. 2010 7th IEEE Working Conf. Mining Software Repositories*,

Cape Town, South Africa, 2010, pp. 114–117.

- [33] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, and M. R. Lyu, Tools and benchmarks for automated log parsing, in *Proc. 2019 IEEE/ACM 41st Int. Conf. Software Engineering: Software Engineering in Practice*, Montreal, Canada, 2019, pp. 121–130.
- [34] P. He, J. Zhu, S. He, J. Li, and M. R. Lyu, An evaluation study on log parsing and its use in log mining, in *Proc.*

2016 46th Annu. IEEE/IFIP Int. Conf. Dependable Systems and Networks, Toulouse, France, 2016, pp. 654–661.

- [35] S. Messaoudi, A. Panichella, D. Bianculli, L. Briand, and R. Sasnauskas, A search-based approach for accurate identification of log message formats, in *Proc. IEEE/ACM 26th Int. Conf. Program Comprehension*, Gothenburg, Sweden, 2018, pp. 167–177.



Jigang Wen received the PhD degree in computer application from Hunan University, China in 2011. He was a research assistant at Department of Computing, The Hong Kong Polytechnic University, China from 2008 to 2010. He is currently an associate professor at Hunan University of Science and Technology, China. His research interests include wireless networks and mobile computing, and high-speed network measurement and management.



Chenying Ma received the BEng degree in computer science and technology from Hunan University, China in 2023. She is currently a master student at Hunan University, China. Her research interests include log parsing and log anomaly detection.



Kun Xie received the PhD degree in computer application from Hunan University, China in 2007. She is currently a professor at Hunan University, China. She has published over 170 articles in major journals and conference proceedings, including journals *IEEE/ACM TON*, *IEEE TMC*, *IEEE TC*, *IEEE TWC*, and *IEEE TSC*, and conferences, including SIGMOD, INFOCOM, ICDCS, SECON, DSN, and IWQoS. Her research interests include network measurement, network security, big data, and AI.



Shiming He received the BEng degree in information security and the PhD degree in computer science and technology from Hunan University, China in 2006 and 2013, respectively. She is currently an associate professor at School of Computer and Communication Engineering, Changsha University of Science and Technology, China. Her research interests include machine learning, data analysis, and anomaly detection.



Shuyu Pei received the PhD degree in computer science and technology from Hunan University, China in 2024. She is now a postdoctoral researcher at College of Computer Science and Electronics Engineering, Hunan University, China. Her research interests include probabilistic data structures and streaming data anomaly detection.



Yuxiang Chen received PhD degree in computer application from Hunan University, China in 2021. He worked as a postdoctoral researcher at Department of Computing, Hunan University, China from July 2021 to Mar. 2023. He is currently an assistant professor at Hunan University of Science and Technology, China. His research interests include network monitoring, network security, big data, and AI.



Wei Liang received the PhD degree in computerscience and technology from Hunan University, China in 2013. He was a postdoctoral researcher at Lehigh University, USA from 2014 to 2016. He is currently a professor and the dean at School of Computer Science and Engineering, Hunan University of Science and Technology, China. He has authored or coauthored more than 140 journal/conference papers, and his research interests include blockchain security technology, networks security protection, embedded system and hardware IP protection, fog computing, and security management in Wireless Sensor Networks (WSN).



Quan Feng received the BEng degree in information and computing science and the MEng degree in computer science and technique from Nanchang Hangkong University, China in 2008 and 2011, respectively, and the PhD degree from Nanjing University of Aeronautics and Astronautics, China in 2022. Currently, he is the head of Machine Learning Research Laboratory, Hunan Vanguard Group Co. Ltd. His research interests are mainly in machine learning and pattern recognition.